ARAVANTA CLOUD OS (ACOS)

Full Project Context, Architecture Specification & Codebase Intelligence

Document Version	Analysis Date	Verification Level	Target System
2.0.0 (Deep Audit)	October 05, 2026	VERIFIED_LOCAL	FastAPI / React Monorepo

CORE GOVERNING PRINCIPLE (AGENTS.MD INVARIANT)
Aravanta Cloud OS is a real cloud control plane. Every service shown in the UI must either:
A) Manage a genuinely provisioned, usable resource at a real provider or local engine, OR
B) State clearly why it cannot (missing provider, credentials, or infrastructure).

The control-plane database is a METADATA REGISTRY, not infrastructure. Fabricated data, invented metrics, and fake telemetry are strictly forbidden. State transitions require cryptographic or provider evidence.

1. Executive Summary & Project Discovery

Aravanta Cloud OS (ACOS) is an open, sovereign cloud control plane platform designed to provide a unified management layer across heterogeneous compute, storage, serverless, database, and event-driven resources. Unlike conventional cloud dashboards that act merely as shallow wrappers or cosmetic registries, ACOS is built around a strict 'Real Control Plane' philosophy defined in AGENTS.md.

System Attribute	Verified Specification
Project Name	Aravanta Cloud OS (ACOS)
Repository Layout	Monorepo (Backend, Frontend, CLI, DevOps, Documentation)
Architecture Pattern	Decoupled SPA Frontend + RESTful API Backend + Background Reconciler
Primary Backend Engine	Python 3.12+ / FastAPI / SQLAlchemy 2.0 / Pydantic v2
Primary Frontend Engine	React 18 / Vite / TypeScript / Tailwind CSS / Lucide Icons
Database Support	SQLite (development & local testing) / PostgreSQL (production)
Encryption Standard	AES-256-GCM authenticated encryption + HKDF master key derivation (ArvVault)
Current Verification Level	VERIFIED_LOCAL (Local Docker Engine & SQLite verified)

2. Technology Stack & Key Dependencies

- **Backend Stack:** FastAPI (ASGI async), SQLAlchemy 2.0, Pydantic v2, Uvicorn, SQLite/PostgreSQL, Alembic.
- **Security & Cryptography:** Python Cryptography library (AES-256-GCM, HKDF, PBKDF2), PyJWT authentication tokens.
- **Cloud SDKs & Engines:** Boto3 (AWS), google-cloud-storage (GCP), azure-storage-blob (Azure), docker-py (Local Docker).
- **Frontend Web Stack:** React 18, Vite 5, TypeScript (strict mode), Tailwind CSS, Lucide React, Axios client.
- **DevOps & Orchestration:** Docker, Docker Compose, Kubernetes manifests, Helm charts, Terraform multi-cloud modules.

3. Workspace Directory Map

Directory Path	Subsystem	Functional Responsibility
backend/	Core API Server	FastAPI application, routers, models, services, core utilities, and database engines.
backend/app/core/	Core Infrastructure	Global config, state guard invariants, database setup, audit logger, and telemetry.
backend/app/models/	Data Models	SQLAlchemy ORM models representing registry entities across all cloud services.
backend/app/services/	Domain Services	ArvCompute, ArvDB, ArvStorage, ArvFunctions, ArvVault, ArvEvents, Reconciler, IAM.
frontend/	Web Dashboard	React 18 single page application, components, view pages, auth context, and styling.
cli/	Operator CLI	Python CLI tools (Typer/Click) for managing ACOS instances, providers, and resources.
docker/	Containerization	Dockerfiles for backend and frontend, docker-compose orchestration environments.
kubernetes/	K8s Orchestration	Manifests for deployment, services, ingress, configmaps, and statefulsets.
terraform/	IaC Provisioning	Terraform modules for bootstrapping AWS, GCP, and Azure control plane infrastructure.
monitoring/	Observability	Prometheus metrics collection configs, Grafana dashboard definitions, and alert rules.
docs/	Documentation	Architecture specifications, verification runbooks, verification evidence, and guides.
tests/	Test Suite	Pytest unit tests, integration tests, security test cases, and provider mock tests.

4. Backend Architecture & Service Catalog

- **ArvCompute:** Manages VMs and containers via Docker local driver or AWS/GCP/Azure cloud provider drivers.
- **ArvDB:** Provisions PostgreSQL, MySQL, and Redis instances with automated backups and vault-injected credentials.
- **ArvStorage:** Object storage abstraction with S3/GCS/Azure drivers and local filesystem storage support.
- **ArvFunctions:** Serverless function execution engine for ephemeral Python and Node.js code snippets.
- **ArvVault:** Zero-plaintext key-value secret vault powered by AES-256-GCM and HKDF master key derivation.
- **ArvEvents:** Asynchronous message queues, topic subscriptions, Dead Letter Queues (DLQ), and exponential retry.
- **Cloud Providers:** Manages provider credentials with live authentication validation (AWS STS, Docker socket).
- **Background Reconciler:** Periodic loop syncing provider state into registry, handling drift and external deletions.

5. State Machine & Invariants Architecture

INVARIANT 2: EVIDENCE-GATED STATE TRANSITIONS

Only a verified provider driver or the Reconciler may transition a resource to RUNNING, AVAILABLE, ACTIVE, or SUCCESS. Transitions are strictly enforced through backend/app/core/state_guard.py. Every transition requires three mandatory parameters: provider_resource_id, state_source, and observed_at.

Lifecycle State	Allowed Next States	Required Evidence / Condition
AWAITING_PROVIDER_SETUP	PROVISIONING, FAILED	Provider connection not yet configured or authenticated.
PROVISIONING	STARTING, RUNNING, FAILED	Asynchronous job dispatched to provider API with idempotency key.
RUNNING / AVAILABLE	STOPPING, DELETING, MISSING	Provider API confirms running status; valid provider_resource_id verified.
STOPPED	STARTING, DELETING	Resource confirmed halted at provider without destruction of storage.
DELETING	DELETED, FAILED	Destruction request acknowledged by provider.
DELETED	Terminal State	Resource confirmed destroyed at provider; tags verified.
MISSING_AT_PROVIDER	DELETING, UNKNOWN	Reconciler detected resource was deleted or drifted out of band.
UNKNOWN	Re-evaluated on next sync	Provider API temporarily unreachable; last known state shown with age.

6. Database Schema & Data Models

Model / Table Name	Primary Columns & Types	Relationships & Constraints
users	id (UUID), email (String, Unique), hashed_password, role_id	Foreign key to roles.id; One-to-many with audit logs.
roles	id (UUID), name (String, Unique), permissions (JSON)	Many-to-one with users.
compute_instances	id (UUID), name, provider, provider_resource_id, actual_state	Unique constraint on provider_resource_id.
database_instances	id (UUID), name, engine, actual_state, host, port	Foreign key to vault_secrets.id for credentials.
storage_buckets	id (UUID), name (Unique), provider, region, actual_state	One-to-many with storage_objects.
storage_objects	id (UUID), bucket_id, object_key, size_bytes, etag	Foreign key to storage_buckets.id.
functions	id (UUID), name, runtime, handler, actual_state	One-to-many with function_executions.
vault_secrets	id (UUID), key (Unique), encrypted_value, nonce, salt	Zero plaintext stored; AES-256-GCM cipher payload.
event_queues	id (UUID), name (Unique), message_count, dlq_id	Self-referential foreign key for Dead Letter Queue.
cloud_providers	id (UUID), provider_type, name, credentials_vault_id	Foreign key to vault_secrets.id for encrypted creds.

Model / Table Name	Primary Columns & Types	Relationships & Constraints
audit_logs	id (UUID), user_id, action, target_resource, timestamp	Immutable append-only trail; Foreign key to users.id.

7. API & Route Inventory

Service	Endpoint Path	HTTP Method	Auth Requirement
Auth / IAM	/api/v1/auth/login	POST	Public (OAuth2 Password)
Auth / IAM	/api/v1/auth/me	GET	Bearer JWT (Any User)
Auth / IAM	/api/v1/auth/register	POST	Admin JWT Required
Compute	/api/v1/compute/instances	GET / POST	Bearer JWT (Compute:Read/Write)
Compute	/api/v1/compute/instances/{id}	GET / DELETE	Bearer JWT (Compute:Read/Write)
Compute	/api/v1/compute/instances/{id}/start	POST	Bearer JWT (Compute:Operate)
Compute	/api/v1/compute/instances/{id}/stop	POST	Bearer JWT (Compute:Operate)
Database	/api/v1/databases	GET / POST	Bearer JWT (DB:Read/Write)
Database	/api/v1/databases/{id}	GET / DELETE	Bearer JWT (DB:Read/Write)
Storage	/api/v1/storage/buckets	GET / POST	Bearer JWT (Storage:Read/Write)
Storage	/api/v1/storage/buckets/{name}/objects	GET / POST	Bearer JWT (Storage:Read/Write)
Functions	/api/v1/functions	GET / POST	Bearer JWT (Functions:Read/Write)
Functions	/api/v1/functions/{id}/invoke	POST	AUDIT ALERT: Unauthenticated
Vault	/api/v1/vault/secrets	GET / POST	AUDIT ALERT: Unauthenticated
Vault	/api/v1/vault/secrets/{key}	GET / DELETE	AUDIT ALERT: Unauthenticated
Events	/api/v1/events/queues	GET / POST	Bearer JWT (Events:Read/Write)
Events	/api/v1/events/queues/{name}/messages	POST / GET	Bearer JWT (Events:Read/Write)
Providers	/api/v1/providers	GET / POST	Admin JWT Required
Providers	/api/v1/providers/{id}/test	POST	Admin JWT Required
Audit	/api/v1/audit/logs	GET	Admin JWT Required

8. Comprehensive Security Audit Findings

CRITICAL: HARDCODED CREDENTIALS DETECTED
Sensitive credentials and secret keys were detected in source files:
- Sensitive credential detected in backend/app/core/config.py (fallback SECRET_KEY default)
- Sensitive credential detected in backend/app/main.py (default superuser seed credentials)
- Sensitive credential detected in main.py (root seed credentials)
- Sensitive credential detected in backend/app/services/arvdb/router.py (fallback database connection string)
Remediation: Remove all hardcoded default fallbacks. Force system failure on startup if mandatory env vars are missing.

HIGH: UNAUTHENTICATED CODE EXECUTION & VAULT ACCESS
1. POST /api/v1/functions/{id}/invoke lacks authentication, allowing arbitrary unauthenticated code execution.
2. GET/POST /api/v1/vault/secrets lacks authentication, exposing sensitive secret retrieval to anonymous users.
Remediation: Enforce Depends(get_current_active_user) across all service router endpoints.

9. Test Suite Execution & Verification Results

Test Suite Component	Observed Outcome	Verification Evidence & Details
Backend Pytest (python -m pytest tests -q)	70 Passed, 4 Failed	Core functionality, vault crypto, state guard, and compute pass. 4 failures caused by field mismatches.
Frontend Typecheck (npm run typecheck)	0 Errors (PASSED)	Complete TypeScript type safety confirmed across all components and API clients.
Frontend Production Build (npm run build)	PASSED (2.84 MB)	Successfully compiled into dist/ directory with zero build warnings.

10. Developer Setup & Onboarding Guide

Execute the following commands from the root directory to run the full environment locally:

```
  # 1. Setup Environment
cd c:\Users\SHUBHAM\Desktop\acos
# Ensure SECRET_KEY and DATABASE_URL are set in .env

# 2. Launch Backend API Server
cd backend
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

# 3. Launch Frontend Web App
cd ../frontend
npm install
npm run dev

# Open: http://localhost:5173 (App) | http://localhost:8000/docs (API)
```